

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1007	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Sk. Parvez Ahamed	Reg. No.:	192411084
Date:	25/08/2026	Duration:	60 minutes

Code of Conduct

I __Sk. Parvez Ahamed (192411084)__ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __Sk. Parvez Ahamed__

Annexure A – CI/CD Pipeline Design Exercise

Blockchain-Based Academic Credential Verification Platform

1. Introduction

The Blockchain-Based Academic Credential Verification Platform is a software system that enables educational institutions to issue digital academic credentials and allows students, employers, universities, and other authorized users to verify them. Since the platform handles academic records and blockchain transactions, the development process requires frequent code integration, automated testing, security checks, reliable deployment, and rollback capabilities. A CI/CD pipeline is therefore designed to automate the process from developer code commit to production deployment while maintaining software quality, security, and reliability.

2. Project Requirements and CI/CD Needs

The CI/CD pipeline should support the following requirements:

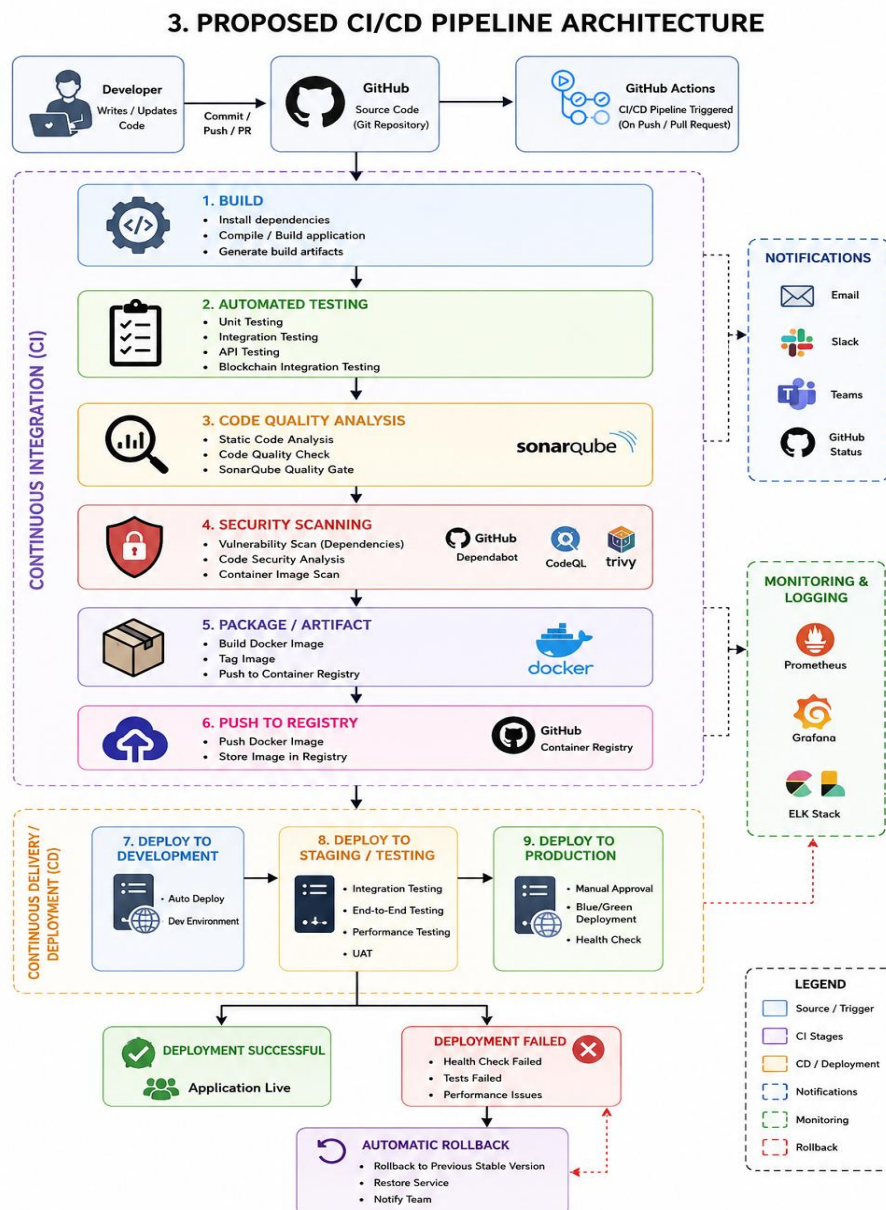
- Automatic integration of code changes.
- Automated compilation/build of the application.

- Automated unit and integration testing.
- Code quality and static analysis.
- Security and dependency vulnerability scanning.
- Creation of deployable application artifacts.

The pipeline should ensure that only tested and approved code reaches production.

3. Proposed CI/CD Pipeline Architecture

Pipeline Flow



4. CI/CD Pipeline Stages

The proposed pipeline consists of the following stages:

Stage 1 – Source Code Management

Developers maintain the application source code in GitHub.

Developers work using feature branches and create pull requests before merging changes into the main branch.

Activities:

- Code commit
- Branch creation
- Pull request
- Code review
- Merge into main branch

Tool: Git + GitHub

Stage 2 – Continuous Integration Trigger

A push or pull request automatically triggers the CI pipeline using GitHub Actions.

The pipeline verifies that the new code can be built and tested before it is merged or deployed.

Tool: GitHub Actions

Stage 3 – Build

The application is compiled and dependencies are installed.

For example:

Install dependencies



Compile application



Build backend/frontend



Generate build artifacts

If the build fails, the pipeline stops and the developer is notified.

Stage 4 – Automated Testing

Automated tests are executed to verify application functionality.

Testing includes:

- Unit testing
- Integration testing
- API testing
- Blockchain integration testing
- Credential verification testing
- Security-related tests

If critical tests fail, deployment is stopped.

Stage 5 – Code Quality Analysis

The source code is analyzed for coding problems and maintainability issues.

Tool: SonarQube

Checks can include:

- Bugs
- Code smells
- Duplicated code
- Maintainability
- Reliability
- Security issues

The pipeline proceeds only if the configured quality gate is passed.

Stage 6 – Security Scanning

Security checks are performed before deployment.

The pipeline should scan:

- Source code
- Third-party dependencies
- Container images
- Configuration files

Possible tools:

- GitHub Dependabot
- Trivy
- GitHub CodeQL

This is particularly important because the application handles academic credentials and blockchain-related transactions.

Stage 7 – Packaging

After successful testing and quality checks, the application is packaged into a deployable artifact.

A Docker container image can be created.

Application Source



Docker Build



Docker Image



Container Registry

Tools:

- Docker
- GitHub Container Registry

The container image provides a consistent environment between development, staging, and production.

5. Tools and Technologies

Pipeline Stage	Tool/Technology	Purpose
Source Control	Git + GitHub	Store and manage source code
CI/CD	GitHub Actions	Automate pipeline execution
Build	Maven/npm/Docker	Build and package application
Testing	JUnit/Jest/Postman	Automated application and API testing
Code Quality	SonarQube	Static code analysis and quality gate
Security	CodeQL/Dependabot/Trivy	Vulnerability and security scanning
Containerization	Docker	Create consistent deployable containers

Pipeline Stage	Tool/Technology	Purpose
Container Registry	GitHub Container Registry	Store Docker images
Deployment	Kubernetes	Deploy and manage containers
Monitoring	Prometheus + Grafana	Monitor application performance
Notifications	GitHub Actions/Email/Slack Notify build and deployment results	

Justification

- GitHub is selected because it provides source-code management, pull requests, code reviews, and integration with GitHub Actions.
- GitHub Actions is suitable because it can automate build, testing, security, packaging, and deployment within the same repository.
- Docker provides environment consistency and prevents differences between development and production environments.
- Kubernetes can be used to manage application containers, scaling, service availability, and deployment strategies.
- SonarQube helps maintain code quality by identifying bugs, vulnerabilities, and code smells.

6. Automated Testing Strategy

Automated testing is performed at multiple stages of the pipeline.

Unit Testing

Tests individual functions and components.

Examples:

- User authentication function.
- Credential validation function.
- Credential ID generation.
- Credential status checking.
- Hash generation and comparison.

Purpose: Detect coding errors early.

Integration Testing

Tests communication between application components.

Examples:

- Application ↔ Database
- Application ↔ Blockchain
- Application ↔ Authentication service
- Application ↔ Credential verification service

Purpose: Ensure that different components work correctly together.

API Testing

API endpoints are automatically tested.

Examples:

POST /login

POST /credentials

GET /credentials/{id}

POST /verify

POST /revoke

The tests verify correct responses, authentication, authorization, and error handling.

Blockchain Testing

Blockchain-related operations should be tested before production deployment.

Tests include:

- Credential hash creation.
- Blockchain transaction submission.
- Transaction confirmation.
- Credential verification.

- Tamper detection.
- Credential revocation.

A test blockchain/network should be used in CI instead of making real production blockchain transactions.

Regression Testing

Previously implemented functionality is automatically tested whenever new code is introduced.

This ensures that a new feature does not break existing functions such as:

- Login
- Credential issuance
- Verification
- Revocation

7. Deployment Strategy

The platform should use three environments.

7.1 Development Environment

Used by developers during active development.

Developer → GitHub → CI → Development

Purpose:

- Feature development
- Initial testing
- Debugging
- Integration testing

7.2 Staging/Testing Environment

The staging environment should closely resemble production.

After successful CI checks, the application is deployed to staging.

Testing includes:

- Integration testing

- End-to-end testing
- Security testing
- Performance testing
- User acceptance testing

Only successfully validated builds proceed toward production.

7.3 Production Environment

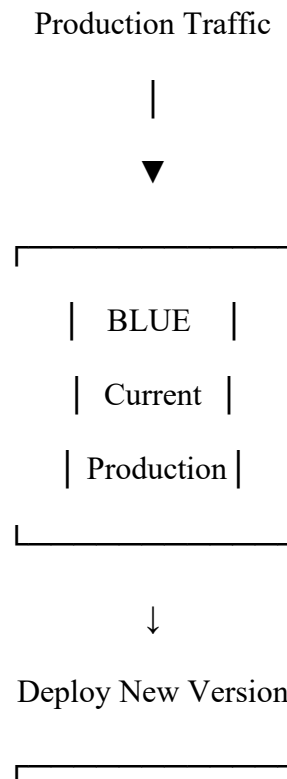
The production environment contains the live application used by real users.

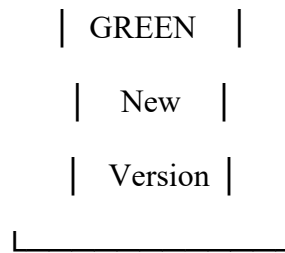
Production deployment should occur only after:

- Build succeeds
- Automated tests pass
- Security checks pass
- Code quality gate passes
- Staging validation succeeds
- Required approval is obtained

8. Deployment Strategy – Blue/Green Deployment

A Blue/Green deployment strategy can be used for production.





The new version is deployed to the Green environment and tested before traffic is switched from Blue to Green.

If the new version works correctly, traffic is redirected to Green.

If a serious problem occurs, traffic can immediately be returned to Blue.

This provides a fast rollback mechanism with minimal downtime.

9. Security Measures

Security should be integrated throughout the CI/CD pipeline.

Source Code Security

- Use protected main branches.
- Require pull-request reviews.
- Prevent direct unauthorized changes to production code.

Secret Management

Sensitive information such as:

- Database passwords
- Blockchain private keys
- API keys
- Authentication secrets

should not be stored directly in source code.

They should be stored using secure secret-management facilities such as GitHub Secrets or a dedicated secret manager.

Dependency Security

Third-party libraries should be regularly scanned for known vulnerabilities.

Container Security

Docker images should be scanned before deployment using tools such as Trivy.

Access Control

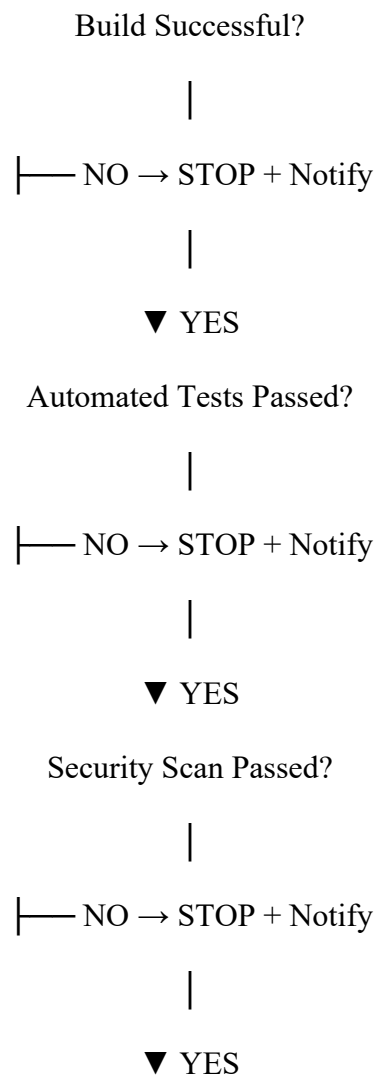
Only authorized users should be able to:

- Approve production deployment.
- Modify pipeline configuration.
- Access production credentials.
- Perform administrative operations.

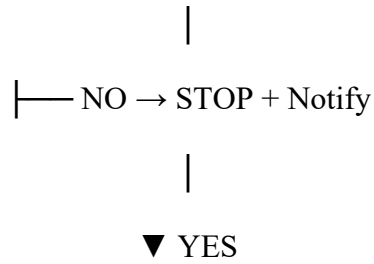
10. Quality Gates

The pipeline should stop when important quality requirements are not satisfied.

Example quality gates:



SonarQube Quality Gate Passed?



Deploy to Staging

This prevents defective or insecure software from reaching production.

11. Rollback Mechanism

If a production deployment causes problems, the pipeline should provide a rollback mechanism.

Rollback can be triggered when:

- Application health checks fail.
- Critical errors occur.
- Verification service becomes unavailable.
- Blockchain integration fails.
- Production tests fail.
- Performance becomes unacceptable.

With Blue/Green deployment, traffic can be switched back to the previous stable version.

New Version



Health Check Failed



Rollback Triggered



Previous Stable Version



Production Restored

The previous stable Docker image should be retained so that it can be redeployed when necessary.

12. Notifications and Monitoring

The pipeline should notify the development team about important events.

Notifications should be generated for:

- Build failure
- Test failure
- Security scan failure
- Quality gate failure
- Staging deployment
- Production deployment
- Rollback

Monitoring tools such as Prometheus and Grafana can be used to monitor:

- Application availability
- CPU and memory usage
- Response time
- Request failures
- Verification-service performance

13. Complete CI/CD Workflow

13. COMPLETE CI/CD WORKFLOW

End-to-End Flow: Code Commit → Production Deployment



14. Justification

The proposed CI/CD pipeline is appropriate for the Blockchain-Based Academic Credential Verification Platform because the system requires high levels of reliability, security, and data integrity.

Automated testing reduces the possibility of defective code reaching production. Code-quality and security scanning help identify vulnerabilities before deployment. Docker provides consistency between environments, while Kubernetes can provide reliable container management.

The use of staging before production reduces deployment risk, while Blue/Green deployment and rollback allow the system to recover quickly if a production problem occurs.

Therefore, the proposed CI/CD pipeline provides an automated and controlled process from code commit → build → testing → quality/security checks → packaging → staging → production → monitoring/rollback.

15. Conclusion

The proposed CI/CD pipeline provides an automated and reliable approach for developing, testing, and deploying the Blockchain-Based Academic Credential Verification Platform. It integrates source code management, automated building, unit and integration testing, code quality analysis, security scanning, Docker-based packaging, and deployment across development, staging, and production environments. The pipeline also includes monitoring, notifications, access control, and rollback mechanisms to reduce deployment risks and maintain system availability. Overall, this CI/CD approach helps ensure that new features and updates are delivered securely, efficiently, consistently, and with high software quality, while protecting the integrity and reliability required for blockchain-based academic credential verification.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25